Single-Leader Replication (Primary-Replica Model)

1. What is Single-Leader Replication?

Single-leader replication means:

```None
One node handles all writes

Multiple replicas handle reads
```

Main node is called:

- Leader / Primary / Master

Other nodes:

- Followers / Replicas / Secondaries

2. Core Idea

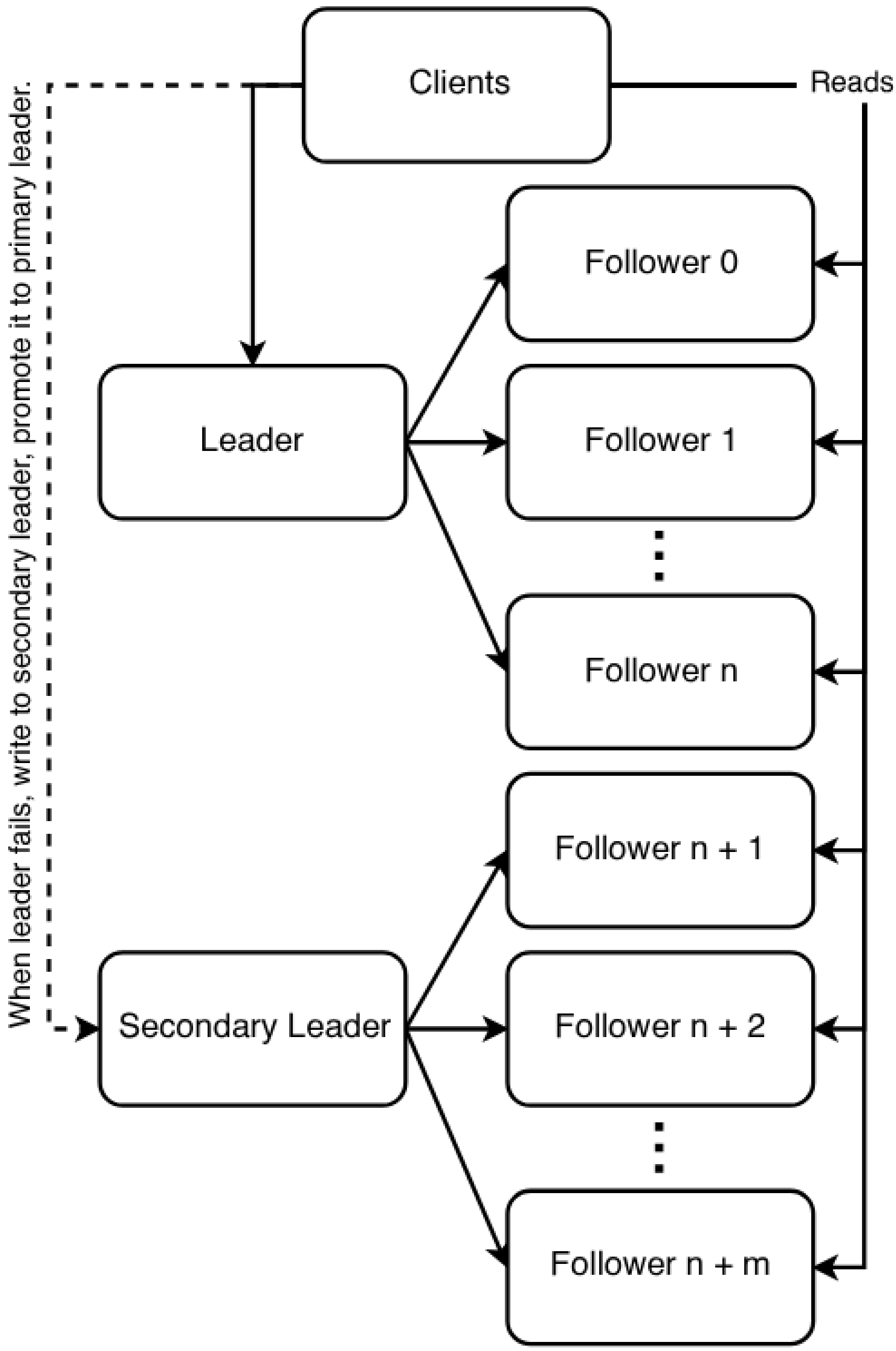
```None
WRITE → Leader only

READ → Leader or Followers
```

Used mainly to scale **reads**, not writes.

3. Architecture

```None
Clients
   |
   | Writes
   v
Primary Leader
   |
   | Replication
   v
Follower 1
Follower 2
Follower 3
```


Clients
Reads
Leader
Follower 0
Follower 1
Follower n
Follower n + 1
Follower n + 2
Follower n + m
Secondary Leader
When leader fails, write to secondary leader, promote it to primary leader.

4. Why Use It?

Single server DB hits limits.

Need:

- More read throughput
- Better availability
- Backup replicas
- Disaster recovery

5. Example

E-commerce product catalog:

```None
Millions browsing products

Few admins updating products
```

Perfect for leader-replica model.

- Reads from replicas
- Writes to leader

6. Real Databases Supporting This

- MySQL
- PostgreSQL
- MongoDB replica sets
- Amazon Aurora

7. Failover Process

If leader crashes:

```None
Replica promoted to new leader
```

Flow:

1. Detect failure
2. Select healthy replica
3. Promote to leader
4. Redirect writes

8. Example Failover

```
Primary Down

Secondary becomes Primary

Old primary returns as replica later
```

9. Main Advantage = Read Scaling

Instead of:

```
1 server handling 1M reads
```

Use:

```
10 replicas handling 100k each
```

10. Main Limitation = Write Bottleneck

All writes still go to one leader.

So:

```None
Leader CPU

Leader disk IOPS

Leader network
```

become bottleneck.

Thus single-leader is **not ideal for write-heavy systems**.

11. Replication Lag

Followers may update after delay.

Example:

```None
Write on leader now

Read from replica instantly

Old data shown
```

This is eventual lag.

12. Consistency Note

Your statement says SQL loses ACID consistency.

More accurate version:

> The primary database can still preserve local ACID transactions. However, distributed replicas may expose stale reads or failover complexities, so globally consistent behavior becomes harder.

ACID on leader ≠ immediate consistency across replicas.

13. When to Use

Choose when:

```None
Read traffic >> Write traffic

Need easy scaling

Need backups

Need failover
```

Examples:

- Product catalog
- Blog platform
- News site
- Analytics reads

14. When to Avoid

Avoid if:

```None
Very high write TPS

Global multi-write system

Strict low-latency writes everywhere
```

Use sharding or multi-leader instead.

15. HLD Interview Talking Points

Discuss:

- Sync vs async replication
- Read/write split routing
- Failover automation
- Replica lag handling
- Backup strategy
- Cross-region replicas

16. Short Revision Notes

```None
Single Leader Replication:


1 leader = writes

many followers = reads


Pros:

read scaling

HA

backup


Cons:

write bottleneck

lag

failover complexity
```

17. One-Line Summary

Single-leader replication scales read traffic by routing writes to one primary node and serving reads from multiple replicas, while trading off write scalability and perfect consistency.